

Vulnerability Detection With Graph Attention Network And Metric Learning

Chunyong Zhang ¹, Bin Liu ², Qi Fan ², Yang Xin ², and Hongliang Zhu ²

¹the School of Cyberspace Security

²Affiliation not available

May 18, 2022

Abstract

Static code vulnerability detection is a critical topic in software security. Existing software analysis methods have a high rate of false positives and false negatives. Researchers are interested in employing deep learning to discover vulnerabilities automatically, thanks to the recent success of deep learning algorithms in other application domains. This paper aims at the problem of insufficient and effective extraction of syntax and semantics, the issue of data imbalance, and the problem of overlapping feature distributions between vulnerable and non-vulnerable samples. We illustrate how to create models in a more principled way. We build GSM, a systematic vulnerability detection model based on Graph Attention Network, Sampling methods, and Metric Learning, one phase for one problem solution. When compared to the state-of-the-art approaches, our method achieves 11.5%, 12.3%, 12.57%, and 7.90% improvement in Precision, Recall, F1-Score, and AUC, respectively. Finally, based on the methods proposed in each stage of this paper, we put forward directions and suggestions for more efficient vulnerability detection tasks in the following research.

Vulnerability Detection With Graph Attention Network And Metric Learning

Chunyang Zhang, Bin Liu, Qi Fan, Yang Xin, and Hongliang Zhu

Abstract—Static code vulnerability detection is a critical topic in software security. Existing software analysis methods have a high rate of false positives and false negatives. Researchers are interested in employing deep learning to discover vulnerabilities automatically, thanks to the recent success of deep learning algorithms in other application domains. This paper aims at the problem of insufficient and effective extraction of syntax and semantics, the issue of data imbalance, and the problem of overlapping feature distributions between vulnerable and non-vulnerable samples. We illustrate how to create models in a more principled way. We build GSM, a systematic vulnerability detection model based on Graph Attention Network, Sampling methods, and Metric Learning, one phase for one problem solution. When compared to the state-of-the-art approaches, our method achieves 11.5%, 12.3%, 12.57%, and 7.90% improvement in Precision, Recall, F1-Score, and AUC, respectively. Finally, based on the methods proposed in each stage of this paper, we put forward directions and suggestions for more efficient vulnerability detection tasks in the following research.

Index Terms—Graph Attention Network, Sampling, Metric Learning, Vulnerability Detection



1 INTRODUCTION

SOFTWARE vulnerability detection has always been a fundamental challenge for system security [1]. Recent statistics show that software vulnerabilities are rapidly increasing; according to the number of software vulnerabilities disclosed by the National Vulnerability Database (NVD) [2], there will be more than 18,000 security vulnerabilities in 2021, including more than 3,000 high-risk vulnerabilities. In addition, software vulnerabilities have also become complex and diverse, with historical vulnerabilities such as "XSS", "RCE", "Ddos" and new vulnerabilities such as "shell-shock", "WannaCry," and "EternalBlue" posing a significant threat to the regular operation of computer software systems. Using various emerging technologies to mine software vulnerabilities and minimize the harm to computer systems is one of the hot issues in cyberspace security.

Traditional vulnerability detection techniques are divided into static, dynamic, and hybrid analysis methods. Static analysis methods mainly include rule-based analysis [3] and middleware-based data control flow analysis [4].

Static-based vulnerability analysis tools such as Coverity¹ and Klocwork². Dynamic analysis methods include symbolic execution analysis [5], [6], fuzzy test analysis methods [7], [8], and dynamic-based vulnerability analysis tools such as libFuzzer³ and AFL⁴. The hybrid technique analysis method combines static and dynamic techniques to achieve a more effective vulnerability detection effect.

With the success of artificial intelligence in image recognition, speech recognition, and anomaly detection, machine learning and deep learning have also been introduced into the research and application of software vulnerability mining. Various vulnerability detection research methods based on machine learning and deep learning have also emerged, and the vulnerability detection problem has been defined as a binary classification problem. Russell et al. [9] utilized a convolutional neural network approach to process the input text sequences and performed vulnerability detection tasks. The two teams of Li. and Lin. are committed to using LSTM or BiLSTM to process the input Token sequence and perform vulnerability detection [10], [11], [12], [13], [14], [15], [16]. Among them, Lin. et al. have used cross-domain theory several times [14], [15]. Research works such as Reveal [17], BGN4VD [18], Devign [19], FUNDED [20], and Poem [21] are dedicated to extracting feature vector representations of code attribute graphs with Gate Graph Neural Network (hereafter, GGNN [22]) and its variants for vulnerability detection.

The neural networks used for training must be able to accommodate non-sequential feature representations, as the original code is more logical and structured than the text language. Even though numerous deep learning models are employed to discover vulnerabilities, there are still

- C. Zhang is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China.
E-mail: chertish_xxd@bupt.edu.cn
- B. Liu is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China, and College of Information and Network Engineering, Anhui Science and Technology University, Bengbu 233030, China;
E-mail: ahstuliubin@bupt.edu.cn
- Q. Fan is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China.
E-mail: sdkjdfq@163.com
- Y. Xin is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China, and State Key Laboratory of Public Big Data, College of Computer Science and Technology, Guizhou University, Guiyang 550025, China.
E-mail: yangxin@bupt.edu.cn
- H. Zhu is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China.
E-mail: zhuhongliang@bupt.edu.cn

Corresponding author: Yang Xin.

1. <https://scan.coverity.com>
2. <https://www.perforce.com/products/klocwork>
3. <http://lvm.org/docs/LibFuzzer.html>
4. <https://lcamtuf.coredump.cx/afl>

significant drawbacks: On the one hand, Convolutional and recurrent neural networks are sequence-based models that can't deal with non-sequential coding characteristics, and these methods typically do not learn the essential features associated with the vulnerabilities. On the other hand, Some graph neural networks (such as GGNN) are unable to store information from both incoming and outgoing edges in a node at the same time, so there is a certain lack of information for the extraction of node feature vectors.

Software vulnerabilities are exceptionally unevenly distributed in the real world. Only a few hundred tens of thousands of functions belong to vulnerable functions in the existing dataset. This is because the collection of vulnerable codes is relatively tricky, and non-vulnerable codes appear more frequently than vulnerable codes. When a neural network model is trained on an unbalanced dataset, the model tends to suffer from non-vulnerable samples, resulting in poor evaluation performance. The performance on the imbalanced dataset tends to have higher Accuracy, but all other metrics are lower than the test performance on the balanced dataset.

In addition, in the feature space, the feature vectors processed by the graph neural network have a deal of overlap. The above work does not aim to widen the class separation between vulnerable and non-vulnerable classes. As a result, they have low accuracy and recall in the real dataset.

Aiming at the following three problems: 1) The syntactic and semantic information cannot be extracted sufficiently and effectively; 2) The problem of data imbalance; 3) The features related to the vulnerability cannot be distinguished. We propose the following workarounds.

First, utilize the code property graph in source code representation since it contains rich syntactic and semantic information, introducing the eigenvectors of the edges in the initialization phase of the node vectors and using multi-layer Graph Attention Network in graph feature vector extraction. It aggregates the information of surrounding nodes using a multi-headed attention mechanism to discern the importance of neighboring nodes. Our ultimate goal is to extract syntactic and semantic information effectively from the code.

Secondly, due to the imbalance between vulnerable and non-vulnerable swatches in the dataset, we adopt a combination of oversampling and undersampling, effectively balancing the dataset and dealing with overlapping samples. This stage also handles indistinguishable samples for the Metric Learning stage.

Finally, since the feature vectors processed by the graph neural network have a large degree of overlap in the feature space, we adopt a Metric Learning method based on the quadruple loss function. The distance between sample feature vectors is typically the goal of Metric Learning. Through training and learning, Metric Learning tries to minimize or restrict the distance between comparable samples and increase the discrepancy between vulnerability and non-vulnerability classes.

On actual datasets, we illustrate the efficacy of our method experimentally. When compared to the current state-of-the-art method Reveal on the dataset chrome_debian, the performances of Accuracy, Precision, Recall, F1-Score, and

AUC are improved by about 4.36%, 11.5%, 12.3%, 12.57%, and 7.90% respectively; on the dataset ffmpeg+qemu, the performance of Accuracy, Precision, Recall, F1-Score, and AUC is improved by about 10.95%, 10.94%, 4.66%, 8.60%, and 10.4%, respectively.

In summary, the following are the contributions of this paper:

- We present GSM, a systematic method for vulnerability detection based on Graph Attention Network, Sampling, and Metric Learning.
- On the public dataset, the efficiency with which of the Graph Attention Network with a multi-head attention mechanism in the extraction of graph feature vectors is verified. The effectiveness of the Metric Learning method based on the quadruple loss function in distinguishing different classes and feature spaces is confirmed.
- The effectiveness of oversampling and undersampling methods is verified on a publicly available actual data set. The performance of the code attribute map for source code characterization is proved to be optimal.
- In the discussion section, the methods proposed in each stage of this paper provide directions and suggestions for the following vulnerability detection task.

We have made our data publicly available on <https://github.com/123zcy/GSM>.

The remaining chapters of this paper are as follows. The background and knowledge of vulnerability detection are presented in Section 2. Our method's pipeline is described in Section 3. The experimental setting and research topics are presented in Section 4. The experimental results are discussed in Section 5. Section 6 discusses the method's limits as well as the possibility of further research. Section 7 is an overview of related work on detecting code vulnerabilities, and Section 8 is a conclusion.

2 BACKGROUND AND KNOWLEDGE

This section introduces the question form, code representation form, word vector embedding method, and commonly used neural network models of deep learning-based vulnerability detection models.

2.1 Vulnerability detection based on deep learning

The vulnerability detection problem based on deep learning is defined as a binary classification problem. For the input source code function, it is determined whether it is a vulnerability after passing through the deep learning model. These approaches learn multiple vulnerability patterns using training datasets to seek vulnerabilities in target static code. The most popular deep learning-based vulnerability detection methods include three stages: data preprocessing, model design, and test evaluation.

Formally, given a code element $\mathbb{C}$ and the corresponding label $l \in [0, 1]$, the goal of a vulnerability detection model is to learn the relative model parameters ϑ , the probability $p(l | \mathbb{C})$ achieves the optimal [17]. Generally, we define as:

$$\vartheta^* = \underset{(\mathbb{C}, l) \in D}{argmax} \prod p(l | \mathbb{C}, \vartheta). \quad (1)$$

2.2 Graph-based code representation

Abstract Syntax Tree (hereafter, AST) is the first step for the code parser to understand the program's basic structure and check syntax errors. It may express the raw code's syntactic structural information as a tree. Representative works representing source code in AST structure include CD-Vuld [14], code2vec [23], and Infercode [24].

A Control Flow Graph (hereafter, CFG) reflects all possible real-time execution flows of a process in a program in the structure of a directed graph, and conditional statements control the processing path. The work of Zhuang. et al. [25], Yu. et al. [26], and Ghaffarian et al. [27] represent the source code in the shape of CFG.

A Data Flow Graph (hereafter, DFG) resembles the transmission and processing of data in a program in the form of a directed graph. A representative work on representing source code structure in DFG is GraphCodeBert [28].

Code Property Graph (hereafter, CPG) [29] comprises an AST, DFG, CFG, and Program Dependency Graph (PDG). Each node and edge on the CPG gives the raw code's general semantic structure as well as supplementary information. In summary, CPG is directed, edge-typed, attribute multigraphs, and each node has at least one attribute to indicate its type. Representative works are represented by code attribute graphs as source codes are BGN4VD [18], Devign [19], and VulSniper [30].

2.3 Word Embedding

The basic idea of word2vec [31] is to use words to predict words. Word2vec is still an encoding method that encodes each word in the language text into a vector, and these vectors can reflect the relationship between these words. It is the most commonly used word embedding method in vulnerability detection tasks [11], [14], [16], [19], [32].

The basic idea of Glove [33] uses the number of co-occurring words in the corpus to obtain the statistical information of the global word, which is the primary method of unsupervised learning to represent the word representation. Glove is also represented by word embeddings in application languages [34], [35].

In addition to the above two word embeddings for code statements within nodes in processing graphs, Doc2vec [36] is also used as word embeddings for program representations [27], [37]. The research [27] also verified that the TF/IDF [38] method is better than Doc2vec in the JAVA language vulnerability detection task.

2.4 Graph Neural Network

Graph Neural Network is commonly utilized in recommender systems, knowledge graph classification, text classification, and other fields due to their remarkable performance and interpretability. With the success of representation learning and word embedding, graph embeddings and graph neural networks have also been employed to static code vulnerability detection tasks. Such graph neural networks mainly include GCN [39], GGNN, GAT [40].

2.5 Metric Learning

The distance between sample feature vectors is generally the goal of Metric Learning. Metric Learning aims to compare a set of feature vectors for similarity through training and learning. Finally, in the embedding space, samples with the same tag are as closely together as possible, while samples with different labels are as far apart as feasible. Common loss functions include contrastive loss function [41], triplet loss function [42] and quadruple loss function [43].

The contrast loss function needs to input positive (negative) sample pairs. By reducing the loss function, the objective is to lower the distance between positive samples while increasing the distance between negative samples. The triplet loss function needs to input a fixed sample, a positive sample, and a negative sample. The goal is to ensure the separation of positive and negative samples in the feature space. In contrast to the triplet loss function, the quadruple loss function requires two negative samples. The goal is to ensure the separation of positive and negative samples and ensure that the distance across classes is continuously reduced. Usually, the quadruple loss function allows the model to learn better representations. Since the feature vectors obtained by the graph neural network have a high degree of overlap in the vulnerability detection task, it is essential to perform Metric Learning. For example, Reveal utilizes representation learning based on the triplet loss function [17].

3 PIPELINE

This section introduces our method's architecture, a vulnerability code detection model that combines Graph Attention Network, Sampling, and Metric Learning; we call it GSM. Fig. 1 shows the detailed process of the GSM method. It consists of three stages: 1) Parse code and graph feature extraction, 2) Sampling 3) Metric learning and classification. GSM takes function slices as input. In the first stage, the function is first parsed into a code attribute graph, then the feature representation of the node is obtained through pre-training and three-layer Graph Attention Network training, and finally, the feature of the code attribute graph is obtained through the readout layer. Express. In the second stage, a combination of oversampling and undersampling is utilized to make the training data in a balanced state. In the third stage, Metric Learning is applied to separate the positive and negative samples to obtain a better graph feature representation. A classifier is used to determine whether the code is a vulnerability. The method expects the training data to be a list of tuples, each tuple containing a function code and corresponding vulnerability label.

3.1 Graph Feature Extraction

This stage aims to convert the function slices in the source code into a feature vector representation of the code property graph. GSM uses Algorithm 1 to extract the feature representation of the code attribute graph. This series of processes is often used in scientific research, but the combination of pre-trained and neural network models used to extract features is slightly different.

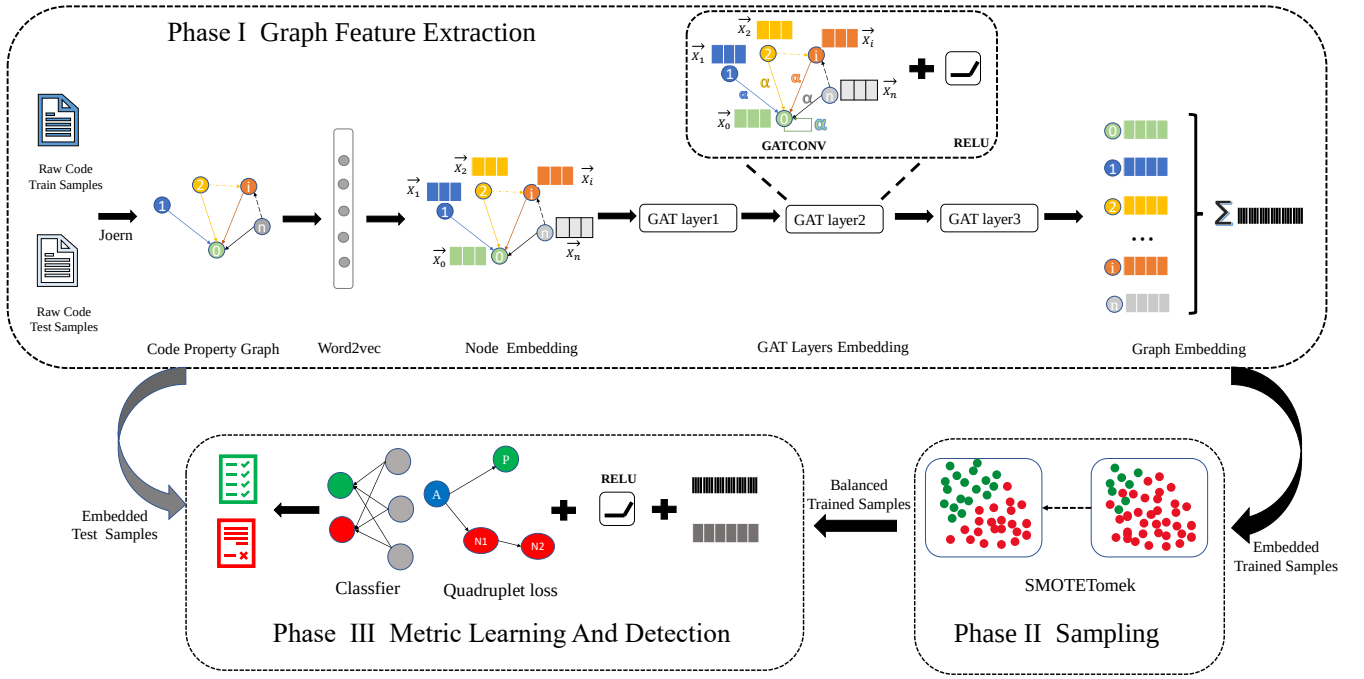


Fig. 1. Framework of GSM

Algorithm 1 Graph Features Extraction (GFE)

```

INPUT: (C)
OUTPUT: (xG)
1 Func GFE(C) :
2   Normalize Fun and Var
3   CPG(V, E) ← Joern(C)
4   X ← Φ
5   For v ∈ V do
6     Pv ← one-hot(vproperty)
7     Sv ← word2vec(vstate)
8     Ev ← one-hot(etype)
9     xv = (Pv || Sv) + (Ev || 0)
10    X ← xv ∪ X
11  End
12  Xl+1 ← ReLU(GAT(G, Xl))
13  xG = ∑v∈V Xl+1
14  return xG

```

Since the raw code is more logical and organizational than the natural text language, this paper first uses the Joern⁵ tool to parse the C language functions. It can generate CPG, which contains many semantic and syntax information in the code. Table 1 and Table 2 show the node information and edge information in CPG, respectively. We can create ASTs, CFGs, and DFGs from node and edge types. Joern creates the nodes of the AST by searching for isASTNode and connecting its children by IS_AST_PARENT edges. In addition, Joern creates nodes of CFG and DFG by searching isCFGNode, and statement nodes are connected through edge types such as FLOWS_TO, CONTROLS, REACHES, USE, and DOM. Each of the above objects has a different in-

TABLE 1
Node information of code property graph

Key	Property	Code	isASTNode	isCFGNode
1	Function	fun	TRUE	
2	FunctionDef	fun (int x1)		
3	RelationalExpression	x1 < 20000	TRUE	
		...		
9	Parameter	int x1		TRUE
10	ExpressionStatement	x1 = 100000		TRUE
11	ReturnStatement	return x1		TRUE
		...		
24	Identifier	x1	TRUE	

terpretation and representation of the semantics and grammatical structure of the code, and separate use cannot make the semantic and grammatical information get the optimal representation. Therefore, this paper uses the code attribute graph connected by all edge types (12 types in total). Fig. 2 shows the code property diagram of a conditional assignment function. Each node contains code statements and attributes elements (e.g., Identifier, AssignmentExpression, ParameterType, Statement, etc.), and the edges of the CPG have directed edges with types. Note that each function has a different edge type because their semantics and syntactic structure are different and may not contain all edges.

In general, the CPG is represented by a two-tuple $G = (V, E)$, where V denotes the collection of nodes, E represents the collection of edges. The graph representation has to deal with the problem of vocabulary explosion - the number of possible identifiers (variables, function names, and constants) in the code is vast. Before word vector embedding, the code is tokenized, the function name is standardized as FUN, and the variable name specification is

5. <https://joern.io>

TABLE 2
Edge Information for Code Property Graphs

Start	End	Type
1	2	IS_FUNCTION_OF_AST
4	5	IS_AST_PARENT
7	8	IS_AST_PARENT
...	...	...
6	10	FLows_TO/CONTROLS
6	24	USE
10	24	DEF
...	...	...

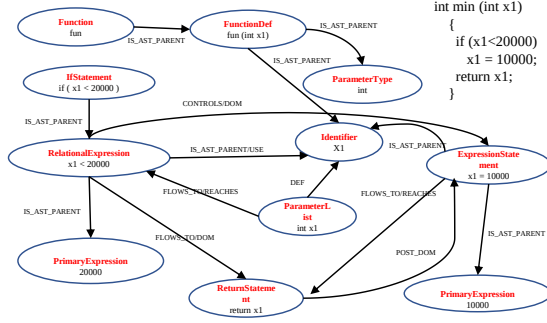


Fig. 2. An example of a CPG.

VAR to reduce data noise caused by customised parameters and function names. It can better maintain the semantic content of the source code. To fully represent the feature vector of the node, let the vector of the attribute element of the node be represented as P_v , and use one-hot [44] to encode the attribute elements of each node. Let the vector representation of the sentence elements be S_v , and use Word2vec to express the word vector embedding for the sentence elements of each node. The edge type also plays a vital role in aggregating node information. Let the vector of edges be represented as E_v , and use one-hot [44] to encode it. Next, splicing the obtained P_v and S_v and aggregating E_v , before aggregation, E_v is filled with 0, and finally get the initial feature vector x_v of each node. x_v is defined as:

$$x_v = (P_v \parallel S_v) + (E_v \parallel 0). \quad (2)$$

The above node feature vectors are obtained independently of other nodes, so the node information is poor. CPG is constructed according to the syntactic and semantic structure between functions, and each node is a sentence fragment, and there are excellent correlations and dependencies between adjacent nodes. To optimally represent the characteristics of a node in the entire code attribute graph, it is necessary to map the information of its neighbor nodes to itself. Considering that each neighbor node has different weights for its influence, we use a Graph Attention Network with a multi-head attention mechanism; the aggregation effect of the attention mechanism on node information has been demonstrated in many studies [27], [34], [45], [46], [47], [48]. Besides, the attention mechanism can deal with latent noise features. The above studies are all different. The attention mechanism is added to the model, and good results are achieved.

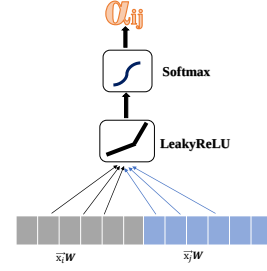


Fig. 3. The operation process of α_{ij} .

Deep layer graph neural networks do not achieve ideal results on most datasets, but single layer graph neural networks do not adequately aggregate node information sufficiently, so in this paper, three layers of GAT are used to extract information from the nodes, and we pre-train each layer of the graph attention network in a supervised manner. Each layer of GAT takes the node vector and the entire graph as input, and there are multiple attention mechanisms between each node and its neighbors, α is calculated by a single-layer feed-forward neural network. The general steps of graph attention network for node update using multi-head attention mechanism are as follows.

The first step is a linear transformation,

$$z_i^{(l)} = W^{(l)} x_i^{(l)}. \quad (3)$$

where $W \in R^{F \times F}$ is the linear transformation weight matrix.

The second step calculates the attention score between paired nodes,

$$e_{ij}^{(l)} = \text{LeakyReLU}(\vec{a}^{(l)T} (z_i^{(l)} \parallel z_j^{(l)})). \quad (4)$$

where $\parallel$ represents a connection, LeakyReLU is a nonlinear activation function, $\vec{a}$ is a learnable weight vector.

The third step is normalized by the softmax function to gain the attention weights,

$$\alpha_{ij}^{(l)} = \frac{\exp(e_{ij}^{(l)})}{\sum_{k \in N(i)} \exp(e_{ik}^{(l)})}. \quad (5)$$

where α_{ij} is the attention weight from node j to node i , and $N(i)$ indicates the collection of neighbors to node i . Figure 3 shows the process of calculating the attention weights.

The fourth step sums all neighbor nodes and splices K independent attention mechanisms,

$$x_i^{(l+1)} = \parallel_{k=1}^K \sigma \left(\sum_{j \in N(i)} \alpha_{ij}^k W^k x_j^{(l)} \right) \quad (6)$$

If the Graph Attention Network is in the last layer, then average,

$$x_i^{(l+1)} = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in N(i)} \alpha_{ij}^k W^k x_j^{(l)} \right) \quad (7)$$

In this paper, the first two layers use the $\parallel$ operation, and the third layer uses the averaging operation. After each layer of GAT, a ReLU function is performed to update the

feature vector of the activated node, and the feature vector x_v^1 of the node obtained through the first layer of network is defined as:

$$x_v^{(1)} = \parallel_{k=1}^K ReLU \left(\sum_{j \in \mathcal{N}(i)} \alpha_{vj}^k W^k x_v \right) \quad (8)$$

The feature vector x_v^2 of the node obtained through the second layer network is defined as,

$$x_v^{(2)} = \parallel_{k=1}^K ReLU \left(\sum_{j \in \mathcal{N}(i)} \alpha_{vj}^k W^k x_v^{(1)} \right) \quad (9)$$

The feature vector x_v^3 of the node obtained through the third layer network is defined as,

$$x_v^3 = ReLU \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}(i)} \alpha_{vj}^k W^k x_j^{(2)} \right) \quad (10)$$

The feature vectors of all nodes in the code property graph are aggregated to generate an n-dimensional graph feature vector x_G as the last step of the first stage. Note that the aggregation function is configurable, LSTM, Mean, etc. can be used, and the summation function is used in this paper.

3.2 Oversampling and Undersampling

This stage aims to process the input of imbalanced sample data into balanced samples using sampling techniques. In the actual code world, the positive and negative sample data is highly imbalanced, and the number of benign codes is far greater than vulnerable. If the imbalance is not addressed, an ideological bias is introduced into the model training, which limits the model's predictive performance

Reveal [17] utilizes the Synthetic Minority Oversampling Technique (hereafter, SMOTE) algorithm [49] to sample unbalanced samples; however, the disadvantage is that the generated minority class samples are easy to overlap with the surrounding majority class samples and are challenging to classify, and the data cleaning technology can deal with the overlapping samples. Therefore, it is possible to oversample and then undersample, so this paper adopts the method (hereafter, SMOTETomek) [50], which is a combination of SMOTE and Tomek Links [51]. In addition, another way to deal with the problem of sample imbalance is the sample weight penalty.

First, we use the SMOTE method. For most benign samples, it will randomly delete some negative examples. For a few vulnerable samples, it will synthesize some new positive samples. Next, we use the Tomek Links method to calculate the distance between different categories. The nearest neighbor samples are directly eliminated so that the neighbors' samples belong to the same class so that the classification can be better. After processing the above two methods, we will obtain class-balanced sample data. Fig. 4a depicts the unbalanced original sample, Fig. 4b shows the samples after SMOTE processing, Fig. 4c illustrates the neighboring samples identified by Tomek Links, and Fig. 4d shows the samples after removing the nearest neighbors.

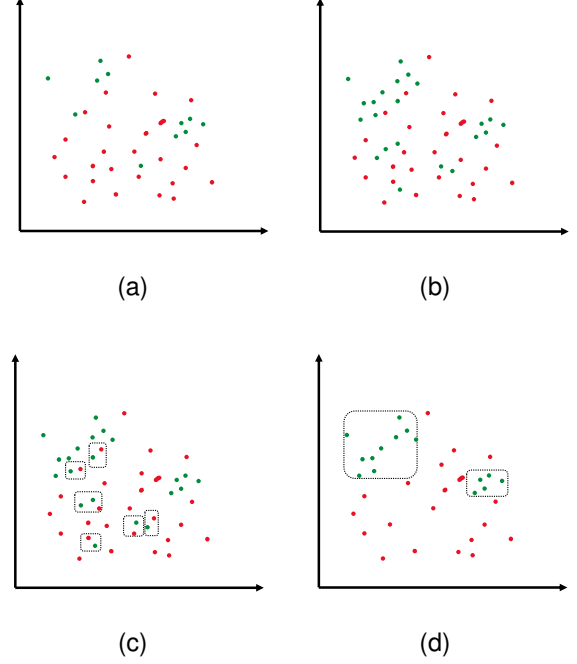


Fig. 4. Sampling process use the SMOTETomek method.

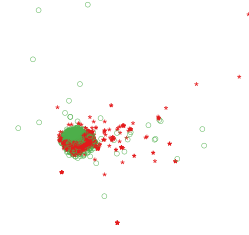


Fig. 5. Embedding space of vulnerable and non-vulnerable samples after Graph Attention Network.

3.3 Metric Learning and Detection

This stage aims to train a Metric Learning model to optimally distinguish overlapping feature vectors and obtain the best classification results. The graph vectors acquired in the first part are reduced in dimensionality and visualized using t-SNE [52]. The feature vectors of the extracted vulnerable and non-vulnerable samples show apparent overlap in the feature space and do not distinguish the positive and negative samples typically. Fig. 5 shows the feature vectors generated by the graph neural network. Positive samples are indicated by a red "*", whereas negative samples are indicated by a green "o".

Suppose there is no apparent difference between the positive and negative samples in the feature vector. This will affect the neural network model's learning effect and limit the model's prediction performance. To maximize the difference between vulnerable samples and non-vulnerable in the projection space. First, use three layers of fully connected layers to map the graph feature vector to a latent feature space to obtain $h(x_G)$, and then adopt a Metric Learning method based on the quadruple loss function [42].

The quadruple loss function is a more complex variant of the triplet loss function [43]. Only the relative distance between positive and negative samples is taken into account by the triplet loss function, which shares fixed samples. When separating negative samples, it will affect the characteristics of fixed samples, affecting the distance control between fixed samples and positive samples. As a result, the quadruple function takes into account the actual disparity between positive and negative instances by combining weak and strong separation. It does not directly affect the eigenvectors of fixed samples while separating negative samples.

In general, a quadruple is represented as (A, P, N_1, N_2) , where (A, P, N_1, N_2) represents a fixed sample, a positive sample and two different negative samples, respectively. The expression of the quadruple loss function is:

$$\mathcal{L}_q = |\mathbb{D}(h(x_A), h(x_P)) - \mathbb{D}(h(x_A), h(x_{N_1})) + \alpha| + |\mathbb{D}(h(x_A), h(x_P)) - \mathbb{D}(h(x_{N_1}), h(x_{N_2})) + \beta|. \quad (11)$$

Where $S_a \neq S_{N_1}, S_a \neq S_{N_2}, S_{N_1} \neq S_{N_2}$. The former term in the formula is called strong separation, and the latter term is called weak separation, α, β is used to define the minimum boundary value, usually set α less than β . $\mathbb{D}(h(x_A), h(x_P)), \mathbb{D}(h(x_A), h(x_{N_1})), \mathbb{D}(h(x_{N_1}), h(x_{N_2}))$ represents the cosine distance between the fixed sample and the positive sample, the fixed sample and the negative sample, and the two negative sample feature vectors, respectively. The expression for $\mathbb{D}$ is as follows:

$$\mathbb{D}(h(x_1), h(x_2)) = 1 - \left| \frac{h(x_1) \cdot h(x_2)}{|h(x_1)| * |h(x_2)|} \right| \quad (12)$$

The larger $\mathbb{D}$ is, the more different the two samples are, and $\mathbb{D}$ is positively correlated with the dissimilarity possibility of the two samples. The quadruple loss function $\mathcal{L}_{II}$ defines whether the two probe samples are the same or not; it is needed that the minimum inter-class distance be larger than the maximum intra-class distance.

Vulnerability detection is a binary classification problem, and we consider the cross-entropy loss function $\mathcal{L}_{ce}$ when calculating the loss function. The definition of the cross loss function is as follows,

$$\mathcal{L}_{ce} = -\frac{1}{N} \sum [y_i \bullet \log(p_i) + (1 - y_i) \bullet \log(1 - p_i)] \quad (13)$$

Where y_i represents the label of the sample S_i , and p_i represents the probability that the sample S_i is predicted to be a positive class.

The final total loss function $\mathcal{L}_{Toa}$ is defined as,

$$\mathcal{L}_{Toa} = \gamma * \mathcal{L}_q + \mathcal{L}_{ce} \quad (14)$$

Where γ is a hyperparameter that, thebe learned by training. The model continuously adjusts the parameters, and the purpose of training is to minimize the total loss function $\mathcal{L}_{Toa}$. Algorithm 2 shows the calculation process of the loss function $\mathcal{L}_{Toa}$.

Algorithm 2 Loss Function calculation process.

```

INPUT:  $\mathbb{M}, \mathbb{S}_{balanced}, x_i, \alpha, \beta, \gamma$ 
OUTPUT:  $\mathcal{L}_{Toa}$ 
1 Func Loss( $\mathbb{M}, \mathbb{S}_{balanced}, x_i, \alpha, \beta, \gamma$ ) :
2   Select  $A, P, N_1, N_2$  from  $\mathbb{S}_{balanced}$ 
3    $h(x_A) \leftarrow \mathbb{M}(x_A)$ 
4    $h(x_P) \leftarrow \mathbb{M}(x_P)$ 
5    $h(x_{N_1}) \leftarrow \mathbb{M}(x_{N_1})$ 
6    $h(x_{N_2}) \leftarrow \mathbb{M}(x_{N_2})$ 
7    $p_i, y_i \leftarrow \mathbb{M}(x_{N_1})$ 
8    $\mathcal{L}_q = |\mathbb{D}(h(x_A), h(x_P)) - \mathbb{D}(h(x_A), h(x_{N_1})) + \alpha|$ 
    $+ |\mathbb{D}(h(x_A), h(x_P)) - \mathbb{D}(h(x_{N_1}), h(x_{N_2})) + \beta|$ 
9    $\mathcal{L}_{ce} = -\frac{1}{N} \sum [y_i \bullet \log(p_i) + (1 - y_i) \bullet \log(1 - p_i)]$ 
10   $\mathcal{L}_{Toa} = \gamma * \mathcal{L}_q + \mathcal{L}_{ce}$ 
11  return  $\mathcal{L}_{Toa}$ 

```

4 EXPERIMENTAL SETUP

4.1 Research Question

We ask the following seven questions to test the efficiency and universality of the strategy provided in this paper.

Research Question 1: How does GSM compare to state-of-the-art methods on real datasets?

Research question 2: How does GSM compare to baseline classification methods on real datasets?

Research Question 3: How do different property graphs affect the vulnerability detection task?

Research Question 4: How does the number of heads of the multi-head attention mechanism affect the vulnerability detection task?

Research Question 5: How does the sampling technique affect vulnerability detection tasks?

Research Question 6: How does the graph neural network affect the vulnerability detection task?

Research Question 7: How effective is metric learning in distinguishing vulnerable samples from non-vulnerable?

4.2 Experimental environment

In terms of the hardware environment, we use a server with model number of Intel(R) Xeon(R) CPU E5-2430 0 @ 2.20GHz and 128 GB RAM. In terms of the software environment, we mainly use the following tool libraries: the C language code parsing tool Joern with version 0.3.1, the deep learning library Pytorch with version 1.9.0, the graph neural network library DGL with version 0.5.1, the version with 0.19.1 of the machine learning library sklearn and version 3.8.1 of the text corpus analysis library Gensim.

The optimal parameter settings involved in our model in each process in this study are shown in the Table 3:

4.3 Dataset

Most of the existing vulnerability detection datasets are collected from CVE⁶ of NVD [2]. Although these vulnerability libraries have high-quality labels, their number is minimal. In addition to this, there are synthetic datasets, like Juliet [53]. However, the datasets are large, the codes generated by their predefined patterns cannot represent the complexity of

6. <http://cve.miter.org>

TABLE 3
An Example of a Table

Model	Parameter	Value
Word2vec	Window size	10
	Vector size	100
GAT	Input dim	169
	Output dim	100/50
	Learning Rate	0.0001
	Number layers	3
	Feat_drop	0.2
	Head_nums	2/4
	Hidden layers	3
	Hidden layers sizes	256,128,256
Metric Learning Model	α	0.5
	β	0.2
	γ	0.001
	Loss function	Quadruplet loss Cross-Entropy Loss
	Optimizer	Adam
	Regularizer	LpRegularizer

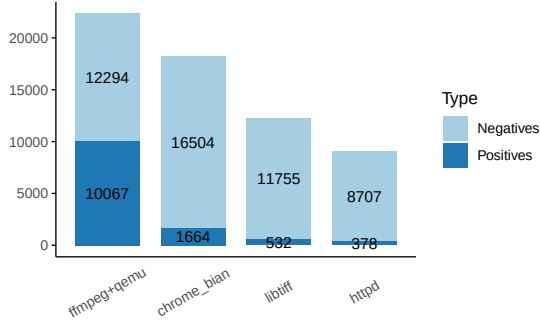


Fig. 6. The proportion of Negatives and Positives.

the actual world, and there is subjectivity. The quality of the dataset will greatly impact the training results of the neural network. To ensure that our method is realistic and effective on various datasets, we choose the dataset chrome_debian reported in Reveal [17], the dataset ffmpeg+qemu reported in Devign, and the dataset libtiff and httpd reported in D2A [54]. These datasets are actual data at the functional level. But the number of vulnerable samples and non-vulnerable in the dataset is imbalanced, as shown in Fig. 6.

The Chrome_debian dataset is obtained from the source code repositories of the open-source projects Linux Debian Kernel and Chromium. For each patch, extract the vulnerability and fixed versions corresponding to the changed source and header files. The version before the patch is as "vulnerability", and marks the patched version as "non-vulnerable".

The ffmpeg+qemu datasets are actual function instances submitted by multimedia libraries and hardware virtualization emulators. Their labels are manually added based on the differences between the submitted information and

the code itself. It is worth mentioning that the vulnerability functions of this dataset account for 45%, which belongs to the category of static code vulnerability detection balanced datasets.

The libtiff and httpd datasets are obtained by D2A from bugs in the submission history of open-source items, and their labels are based on static difference analysis methods between programs.

4.4 Evaluation Indicators

The evaluation metrics and evaluation process are critical to evaluating the model. To avoid the singleness and randomness of index evaluation, various indicators are selected to assess the performance of this method in vulnerability detection applications. We choose Accuracy, Precision, Recall, F1-Score, and AUC, where AUC refers to the area under the ROC curve; this indicator can better evaluate the performance of the imbalanced sample classifier. The remaining indicators' calculating formula is as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (15)$$

$$Precision = \frac{TP}{TP + FP} \quad (16)$$

$$Recall = \frac{TP}{TP + FN} \quad (17)$$

$$F1 - Score = 2 \times \frac{Precision \bullet Recall}{Precision + Recall} \quad (18)$$

Where TP denotes True Positive, FP denotes False Positive, TN denotes True Negative, and FN denotes False Negative.

A dataset should contain two types of vulnerable samples and non-vulnerable samples. We randomly divide all data into training, validation, and test datasets by 8:1:1 and ensure that both training and testing datasets contain vulnerable samples. Because of the instability and randomness of deep learning model testing, we conduct 15 tests for each experimental content, take the average value (retain three decimal places) as the final experimental result, and use the scatter box plot to display the results of these 15 times. Note that the following experiments were performed according to the above settings.

5 RESULTS AND ANALYSIS

This section outlines the methodology and experimental findings for each research question, as well as the implications for future research efforts.

5.1 GSM vs. State-of-the-art Approaches

RQ1: How does GSM compare to state-of-the-art methods on real datasets?

Contrasting methods: 1) VulDeePecker: A bidirectional long-short-term memory network-based approach for vulnerability detection at the sentence level. 2) SySeVR: A Code Snippet-Based Bidirectional Gated Neural Network Vulnerability Detection Method. 3) Reveal: A vulnerability

TABLE 4
Performance comparison between GSM and VulDeePecker, SySeVR and Reveal

Dataset	Methods	Accuracy	Precision	Recall	F1-score
chrome_debian	VulDeePecker	77.133	18.413	19.968	18.944
	SySeVR	78.754	24.987	30.035	26.636
	Reveal	84.083	31.328	63.048	41.677
	GSM-2	88.443	42.806	75.369	54.428
	GSM-4	86.437	37.413	70.411	48.68
ffmpeg+qemu	VulDeePecker	49.114	47.43	67.996	55
	SySeVR	51.307	47.163	50.068	48.19
	Reveal	61.793	56.021	75.483	64.267
	GSM-2	72.708	66.959	80.138	72.866
	GSM-4	73.562	68.223	79.546	73.374

* Bold indicates the best performance.

detection method that combines gated graph neural networks and representation learning. Reveal was the primary contrasting method in this study, and our work was inspired by it.

Datasets: chrome_debian and ffmpeg+qemu.

Experimental results: Table 4 compares the vulnerability detection results of GSM, VulDeePecker, SySeVR, and Reveal under the real data set. The table indicates that no matter whatever data set is utilized, the different GSM approaches function well. Based on the most stable method, GSM-2, in the dataset chrome_debian, the specific performance is about 4.36%, 11.5%, 12.3%, and 12.57% higher than the optimal method Reveal in Accuracy, Precision, Recall, and F1-Score, respectively. In the dataset ffmpeg+qemu, the Accuracy, Precision, Recall, and F1-Score are improved by about 10.95%, 10.94%, 4.66%, and 8.60%, respectively.

Fig. 7a and Fig. 7b show box plots of AUC results for Reveal, GSM-2, and GSM-4 on balanced (sampled) and imbalanced (unsampled) datasets. Fig. 8a and Fig. 8b show the box scatter plots of the results of other indicators on the two datasets for Reveal, GSM-2, and GSM-4, respectively. The above figure indicates that the GSM method is better than Reveal in any data set and indicators. This is due to the combination of the multi-head attention mechanism of the Graph Attention Network and the Metric Learning method, which makes the graph feature vector representation optimal and captures more syntactic and semantic information in the CPG.

5.2 GSM vs. Baseline Approaches

RQ2: How does GSM compare to baseline classification methods on real datasets?

Contrasting methods: 1) SVM [55] is a generalized linear classifier classified according to supervised learning. 2) LR [17] is a machine learning method for solving binary classification problems. 3) RF [15] is a classifier that uses a tree structure to train and predict samples. 4) Adaboost [56] is a robust classifier suitable for combining sampling methods. This paper uses the sampling method, so Adaboost is added to the baseline comparison.

Dataset: chrome_debian and ffmpeg+qemu.

Experimental results: Table 5 shows the performance comparison of GSM and SVM, LR, RF, and Adaboost under the indicators Accuracy, Precision, Recall, F1-Score, and AUC. The table shows that except on the dataset chrome_debian, although the SVM has the highest Accuracy, the Recall and F1-Score are both low. The most stable and optimal method is GSM, which is only about 2% smaller than SVM in Accuracy. Taking GSM-2 as the benchmark, compared with Adaboost in the dataset chrome_debian, the Accuracy, Precision, Recall, F1-Score, and AUC are improved by about 8.01%, 15.47%, 4.37%, 14.98%, and 6.4% respectively; in the dataset ffmpeg+qemu, the Accuracy, Precision, Recall, F1-Score, and AUC are improved by about 12.64%, 12.24%, 6.45%, 10.07%, and 12.20%, respectively.

Fig. 9a and Fig. 9b show the heat maps under the two data sets. It can be observed that the color contrast under the type Baseline and the GSM method, the blue value is low and the red value is high. The quadruple loss function and the cross-entropy loss function in the metric learning method enable the vulnerable samples and non-vulnerable to be distinguished in the feature space, making the final classification effect better.

5.3 CFG vs. DFG vs. CFG_DFG vs. CPG

RQ3: How do different property graphs affect the vulnerability detection task?

Property graph: 1)CFG 2)DFG 3)CFG_DFG 4)CPG

Dataset: chrome_debian and ffmpeg+qemu.

Experimental results: Table 6 shows the code representation performance of different property graphs under other datasets. It is observed from the table that the CPG (FULL) representation of the source code achieves the best performance. The representation of the source code by CFG and DFG is almost the same under various indicators. When CFG and DFG are combined, it is clear that the representation ability has increased, but the improvement effect is not apparent. When using CPG, it can be observed that the representation performance improves significantly. The specific performance is that under the data set chrome_debian, the indicators have increased by about 2.85%, 6.74%, 7.09%, 7.34%, and 4.80%, respectively; under

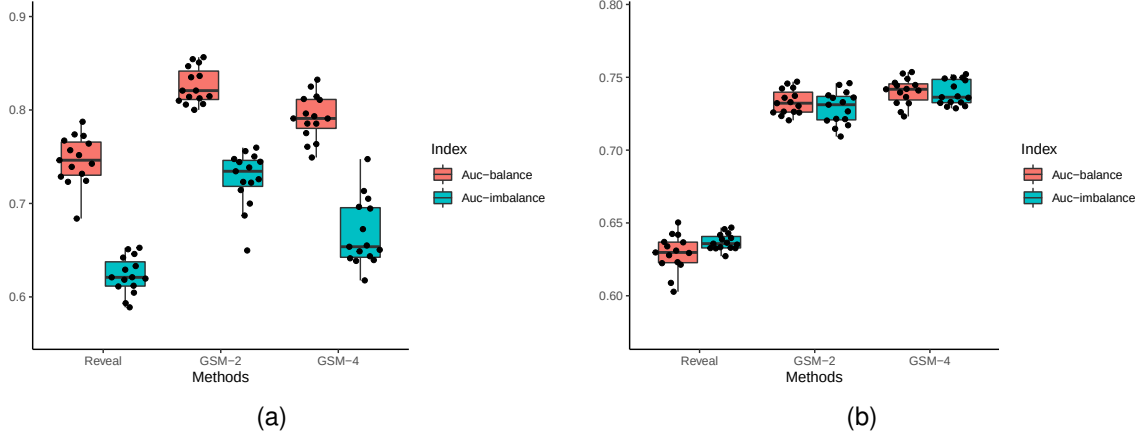


Fig. 7. (a) AUC comparison under Chrome_debian(With Sampling or W/O Sampling). (b) AUC comparison under ffmpeg+qemu(With Sampling or W/O Sampling).

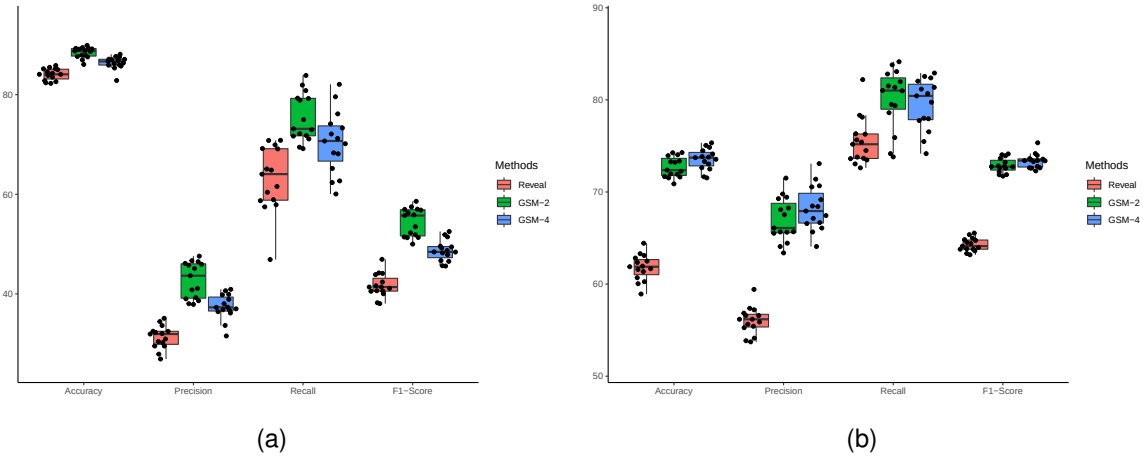


Fig. 8. (a) Performance comparison between Reveal, GSM-2, and GSM-4 under Chrome_debian. (b) Performance comparison between Reveal, GSM-2, and GSM-4 under ffmpeg+qemu.

TABLE 5
Performance comparison between GSM and SVM, LR, RF, and Adaboost

Dataset	Approaches	Accuracy	Precision	Recall	F1-Score	Auc
chrome_debian	SVM	90.207	25.521	2.652	4.796	0.509
	Adaboost	80.438	27.333	71.004	39.445	0.762
	LR	83.482	31.52	70.183	43.48	0.775
	RF	85.399	23.129	25.841	24.38	0.586
	GSM-2	88.443	42.806	75.369	54.428	0.826
	GSM-4	86.437	37.413	70.411	48.68	0.792
ffmpeg+qemu	SVM	60.719	60.426	38.829	47.272	0.589
	Adaboost	60.063	54.718	73.689	62.799	0.611
	LR	63.967	59.414	66.09	62.57	0.641
	RF	57.326	52.975	50.891	51.905	0.568
	GSM-2	72.708	66.959	80.138	72.866	0.733
	GSM-4	73.562	68.223	79.546	73.374	0.74

* Bold indicates the best performance.

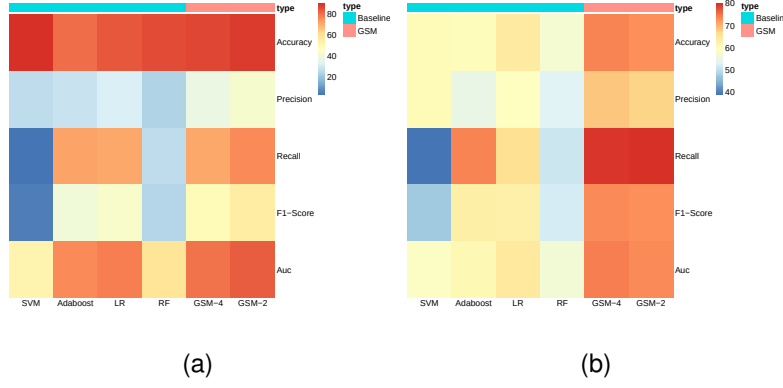


Fig. 9. (a) Performance comparison between GSM, SVM, LR, RF, and Adaboost under chrome_debian. (b) Performance comparison between GSM and SVM, LR, RF, and Adaboost under ffmpeg+qemu.

TABLE 6
Characterization performance comparison of different property graphs

Dataset	Graph_Type	Accuracy	Precision	Recall	F1-Score	Auc
chrome_debian	CFG	85.258	34.167	66.088	44.846	0.766
	DFG	83.116	31.269	64.774	42.032	0.749
	CFG_DFG	85.591	36.065	68.284	47.083	0.778
	CPG(FULL)	88.443	42.806	75.369	54.428	0.826
ffmpeg+qemu	CFG	60.092	54.081	82.976	65.393	0.62
	DFG	56.677	51.801	76.261	61.603	0.582
	CFG_DFG	60.616	55.107	80.684	65.151	0.623
	CPG(FULL)	72.708	66.959	80.138	72.866	0.733

* Bold indicates the best performance.

the dataset ffmpeg+qemu, except that Recall is not optimal, the remaining indicators are improved by 12.09%, 11.85%, 7.47%, and 11% respectively.

Fig. 10a and Fig. 10b show the box-shaped scatter plots of the Precision, Recall, and F1-Score indicators under the two datasets with different attribute graphs. The above two graphs can intuitively show the optimal performance of CPG in source code representation. CPGs are composite graphs of CFG, DFG, AST, and PDG, giving a richer source code representation. CPGs represent more syntactic and semantic information in the source code, which allows the graph neural network model to learn better graph feature representations.

5.4 GSM-1 vs. GSM-2 vs. GSM-4

RQ4: How does the number of heads of the multi-head attention mechanism affect the vulnerability detection task?

Contrasting methods: 1) GSM-1. 2) GSM-2. 3) GSM-4. They indicate the number of head attention mechanisms used by the graph neural network in the GSM method is 1, 2 and 4, respectively.

Dataset: chrome_debian and ffmpeg+qemu.

Experimental results: GSM adopts the Graph Attention Network of multi-head attention mechanism to capture the information importance of nodes better. Table 7 shows the results of using a different number of attention mechanisms

in the attention network in the following figure on the chrom_debian and ffmpeg+qemu datasets. It can be observed from the table that under the chrome_debian dataset, GSM-2 has the best performance, with each index reaching 88.44%, 42.81%, 75.37%, 54.43%, and 82.6%, respectively. Under the ffmpeg+qemu dataset, GSM-1 achieves 85.00% in Recall, GSM-4 has the best results in the remaining indicators, with each indicator reaching 73.56%, 68.22%, 73.374%, and 74.00%, respectively.

Fig. 11a and Fig. 11b reveal the performance of different methods on the two datasets, respectively. Intuitively, GSM-2 and GSM-4 have their advantages in datasets compared to GSM-1. Because GSM-1 adopts the single-head attention mechanism, it is impossible to assign multiple weights to the neighbor node information. In addition, it is not that the more attention mechanisms, the better the performance. Due to the difference in the code of the data set itself, it is necessary to compare the number of attention mechanisms to make the final result optimal.

5.5 With Sampling vs. Without Sampling

RQ5: How does the sampling technique affect vulnerability detection tasks?

Contrasting methods: 1) The SMOTETomek technique balances the dataset under GSM-2. 2) The SMOTETomek technology is not used to balance the dataset under GSM-2.

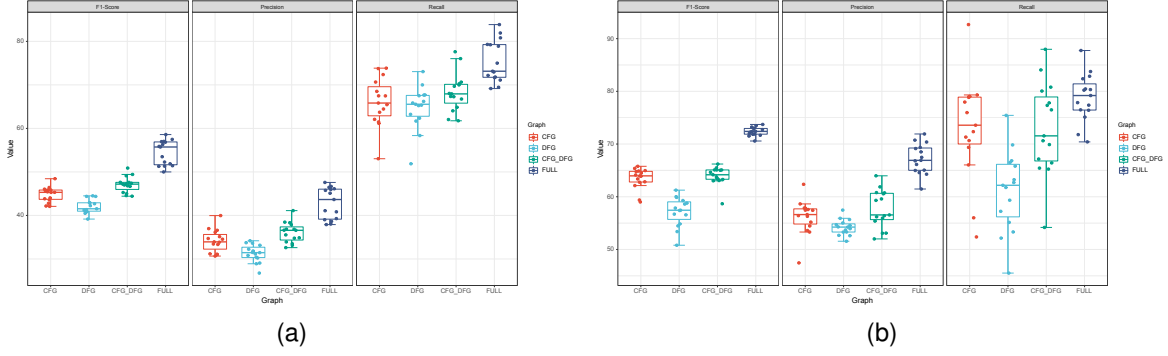


Fig. 10. (a) Performance comparison of different property graphs under Chrome_debian. (b) Performance comparison of different property graphs under ffmpeg+qemu.

TABLE 7
Vulnerability detection performance comparison of different attention mechanisms

Dataset	Methods	Accuracy	Precision	Recall	F1-Score	Auc
chrome_debian	GSM-1	87.505	39.895	67.506	49.985	0.785
	GSM-2	88.443	42.806	75.369	54.428	0.826
	GSM-4	86.437	37.413	70.411	48.68	0.792
ffmpeg+qemu	GSM-1	47.926	46.66	84.997	59.235	0.506
	GSM-2	72.708	66.959	80.138	72.866	0.733
	GSM-4	73.562	68.223	79.546	73.374	0.74

* Bold indicates the best performance.

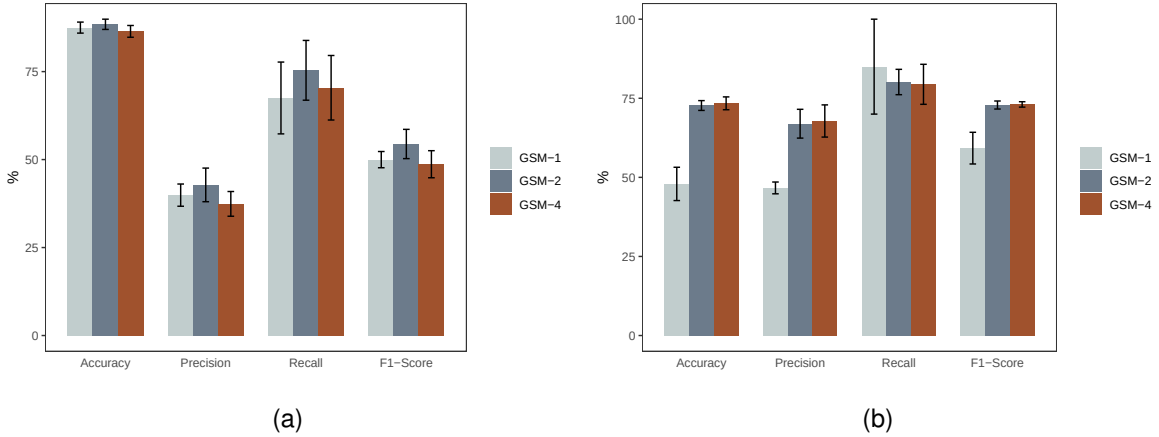


Fig. 11. (a) Performance comparison of GSM-1, GSM-2 and GSM-4 under chrome_debian. (b) Performance comparison of GSM-1, GSM-2 and GSM-4 under ffmpeg+qemu.

Dataset: chrome_debian, ffmpeg+qemu, libtiff, and httpd.

Experimental results: GSM uses SMOTETomek technology to solve the problem of imbalanced data. Table 8 reveals the outcomes of each indicator with and without SMOTETomek technology. It can be seen from Table that under the chrome_debian, libtiff, and httpd datasets, the performance obtained without sampling on the Accuracy is better. This is because these datasets are unbalanced, and the proportion of non-vulnerable functions is much larger than that of vulnerable. However, the ffmpeg+qemu dataset

is similar to the balanced dataset, so the indicators are not much different under the two methods. The performance of the chrome_debian, ffmpeg+qemu, and httpd datasets under the Recall and F1-Score using the SMOTETomek method is better. In addition, the Recall and AUC results obtained by the sampling method are better, so the SMOTETomek method is not used if the task of focusing on Precision, and the SMOTETomek process is used if the task of concentrating on Recall and AUC is used.

Fig. 12 shows the AUC results of the four datasets under the With Sampling and W/O Sampling methods. Except

TABLE 8
Vulnerability detection performance comparison of different attention mechanisms

Dataset	Methods	Accuracy	Precision	Recall	F1-score	Auc
chrome_debian	With Sampling	88.443	42.806	75.369	54.428	0.826
	W/O Sampling	92.23	57.934	49.453	52.441	0.726
ffmpeg+qemu	With Sampling	72.708	66.959	80.138	72.866	0.733
	W/O Sampling	72.413	67.183	78.829	72.38	0.729
libtiff	With Sampling	88.113	24.841	82.368	37.898	0.854
	W/O Sampling	96.422	60.232	49.976	54.278	0.742
httpd	With Sampling	96.444	27.603	68.079	38.663	0.825
	W/O Sampling	98.596	23.254	4.415	6.97	0.522

* W/O Sampling=Without Sampling, Bold indicates the best performance.

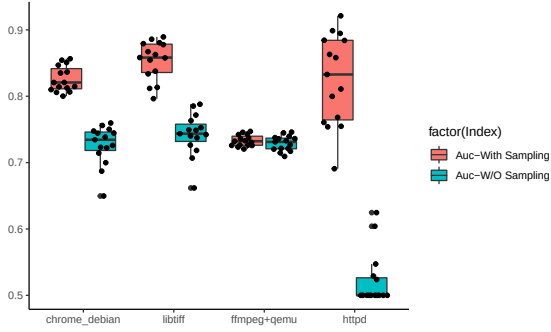


Fig. 12. AUC results With or W/O Sampling method.

for the balanced dataset ffmpeg+qemu, the AUC results obtained by the sampling method for other unbalanced datasets have been significantly improved. According to Table 6, the AUC is increased by 10%, 11.2%, and 30.3%, respectively, under the chrome_debian, libtiff, and httpd datasets. Note that under the httpd dataset, the AUC of the W/O Sampling method is almost close to 50%, which indicates that the method is ineffective, while With Sampling have a good performance, which once again verifies the effectiveness of the sampling method. If you have an imbalanced dataset, use the sampling method, and you will get unexpected results.

5.6 With GAT vs. Without GAT

RQ6: How does the graph neural network affect the vulnerability detection task?

Contrasting methods: 1) Use the Graph Attention Network to update the node feature vector and then sum it up to obtain the graph feature vector, based on the GSM-2 method. 2) Instead of using the Graph Attention Network to upgrade the node feature vector, the node vector sum is directly used to obtain the graph feature vector.

Dataset: chrome_debian and ffmpeg+qemu.

Experimental results: Table 9 shows the comparison of vulnerability detection performance with and without Graph Attention Network. Under the chrome_debian dataset, the GAT method (GSM-2) outperforms the W/O GAT method in every index. The specific performance is

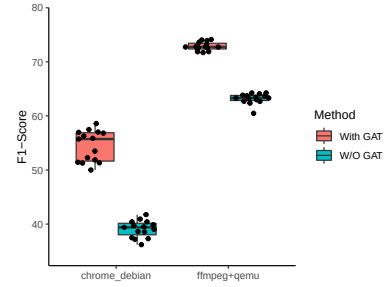


Fig. 13. F1-Score comparison with and without Graph Attention Network under chrome_debian and ffmpeg+qemu datasets.

that the commission under Accuracy, Precision, Recall, F1-Score, and AUC is improved by 5.44%, 13.54%, 16.12%, 15.33%, and 10.3%, respectively. Under the ffmpeg+qemu dataset, W/O GAT shows a higher recall rate, but other metrics are lower than the method of With GAT. Specifically, the performance is reduced by 20.25%, 18.04%, 9.63%, and 17.90% under Accuracy, Precision, F1-Score, and AUC, respectively. Although the W/O GAT method sometimes has a higher recall rate, the performance is lower than that of the With GAT method on the F1-Score metric, as shown in Fig 13. The method of W/O GAT does not aggregate the information of neighbor nodes so that the node information representation is scarce, so the final graph representation loses a lot of global information. However, the method of With GAT utilizes the graph neural network based on the multi-head attention mechanism to aggregate the neighbor node information so that the global information is more abundant. Finally, the graph feature vector is optimally represented.

5.7 With Metric Learning vs. Without Metric Learning

RQ7: How effective is Metric Learning in distinguishing vulnerable samples from non-vulnerable?

Contrasting methods: 1) Based on GSM-2, the quadruple loss function and the cross-entropy loss function are adopted. 2) Based on GSM-2, the triplet loss function is adopted.

Dataset: chrome_debian.

TABLE 9
Vulnerability detection performance comparison with and without GAT

Dataset	Methods	Accuracy	Precision	Recall	F1-Score	Auc
chrome_debian	With GAT	88.443	42.806	75.369	54.428	0.826
	W/O GAT	83.002	29.265	59.244	39.096	0.723
ffmpeg+qemu	With GAT	72.708	66.959	80.138	72.866	0.733
	W/O GAT	52.456	48.923	89.531	63.233	0.554

* W/O GAT=Without GAT, Bold indicates the best performance.

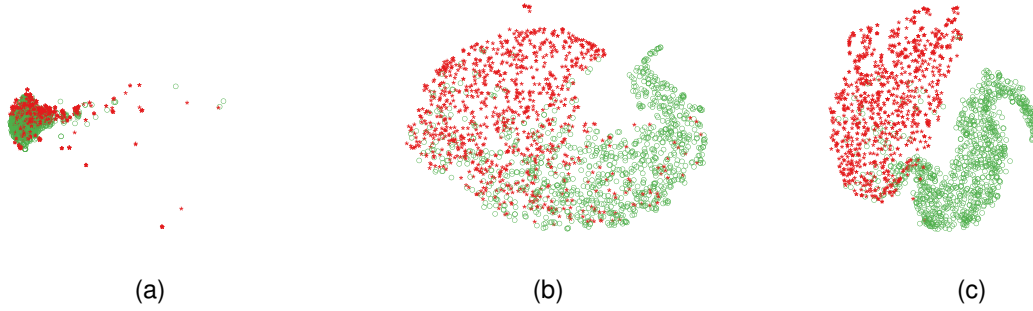


Fig. 14. (a) Feature space distribution before metric learning. (b) Feature space distribution after triplet loss function. (c) Feature space distribution after quadruple loss function and cross-entropy loss function.

Experimental results: GSM adopts the method of Metric Learning to distinguish feature vectors to obtain better graph feature representation and to separate vulnerable samples and non-vulnerable samples to the greatest extent. It can be observed from Fig 14a that after graph feature vector extraction and sampling, the graph feature vectors in the feature space have a great degree of coincidence, and different graph features cannot be distinguished well. The triplet loss function is used in the Reveal method. The spatial representation obtained by training on the graph vector in this paper is shown in Fig 14b. It can be observed that there is a certain degree of discrimination in the feature vector, but there is still a part of overlap between the classes. The quadruple loss function and cross-entropy loss function adopted in this paper map the coincident graph feature vectors to a larger space and distinguish between larger classes, as shown in Fig 14c. Where the red “+” represents positive samples, and the green “o” represents negative samples.

6 DISCUSSION AND FUTURE WORK

First of all, in the graph feature extraction stage, we adopted the Graph Attention Network based on the multi-head attention mechanism. We are not sure whether the 3-layer Graph Attention Network is the best result, but compared with Reveal, GSM has a significant performance improvement in various indicators. In addition, the vulnerability detection task is a graph classification task. In this paper, the edge features are considered when the node vector is initialized and aggregated into the node features. For extracting graph features, we have the following considerations: 1) Can the superposition of Graph Attention Network layers draw on the idea of the residual network [57] or combine with

Jump Knowledge Network [58] to preserve more original node information? 2) Graph classification should aggregate the information of nodes and edges. You can try a Graph Attention Network that pays attention to both node and edge features. 3) Most of the previous works are based on GGNN [17], [18], [19], [20], [21]; what is the performance of the Gated Attention Network (GaAN) [59] using the combination of attention mechanism and gating unit?

Secondly, in the sampling stage, due to the imbalance between the vulnerable and non-vulnerable samples in the dataset, this paper chooses the method SMOTETomek which combines oversampling and undersampling, which solves the problem of overlapping sample data caused by only using the oversampling method SMOTE. In addition to this, there are many sampling methods that can be used, and weight penalty methods can also solve the problem of sample imbalance. However, there is still no work to determine which method performs better in vulnerability detection under imbalanced data.

Finally, we adopt a Metric Learning method based on a quadruple loss function in the Metric Learning stage due to the high overlap of graph feature vectors and the indistinguishability of sample categories. There is a better distinction between classes in the feature space than Reveal. However, there is still a high degree of overlap in the intersection part, so it is necessary to study a metric learning method suitable for vulnerability detection tasks.

This paper provides a model framework for other researchers, and the above problems can theoretically be verified. In the future, we will use the method of adversarial transfer learning to solve the problem of difficulty in sample labeling and use the idea of the adversarial network [60] to find a suitable measurement method to adapt to the difference between different datasets (which can be data

sets in other languages), and finally realize the task of code vulnerability detection across projects or programming languages.

7 RELATED WORK

To improve code analysis, many researchers have explored integrating machine learning with program analysis. In this section, this paper reviews some published work on program code representation and static code vulnerability detection using machine learning or deep learning.

Program code representation. The first job of the vulnerability detection task is to characterize the source code as feature vectors. Yahav et al. [61] proposed code2vec, a method based on path-level attention, which calculates the weight of each path extracted from the AST and then adds the representations of each path according to the weight. Finally, it obtains the representation of the code. Zhang et al. [62] presented an AST-based neural network code representation approach. The method first converts the code into AST, inputs each subtree into the sentence encoder, obtains the representation of each subtree, then goes through RNN and pooling layer, and finally gets the representation of the code. Tal et al. [63] proposed Neural Code Comprehension (NCC), a semantic-based code representation independent of specific programming languages. After inst2vec, the obtained word vector is passed through RNN to get the final code representation.

Vulnerability detection method based on RNN. This method usually first performs lexical analysis on the source code, converts the characters in the source code into Token sequences according to the lexical grammar of the programming language, embeds the word vector into the feature space, and then uses the related methods of RNN for training and detection. VulDeePecker [10] takes function calls as the starting point, selects code gadgets to represent the semantically related code sentence sets, and then selects Bi-LSTM to extract feature vectors. μ VulDeePecker [12] introduced code attention based on VulDeePecker to focus on the local features of the program. It fuses global features as well as local features through Bi-LSTM. The above two methods only consider data dependencies when dividing slices and do not consider the structural information of the source program. As a result, they can only cope with flaws in function calls. Lin et al. all used Bi-LSTM for feature vector training and extraction, and their work all treated the code as a text sequence, so the complex structural information of the code was not considered [14], [15], [16]. Among them, the multi-domain theory is used many times in the research [14], [15].

Vulnerability detection method based on graph neural networks. Usually, to obtain the grammatical and semantic structure of the raw code, it is first processed into a graph structure or tree structure by tools, and the nodes are initialized by the word vector method and then input into the graph neural network for training. Finally, the downstream vulnerability detection task is performed. Devign [19], FUNDED [20], and Poem [21] are all vulnerability detection methods using a gated graph neural network based on multilateral type relationships. These methods consider various edge relationships based on AST, but Poem integrates the

control data flow graph of the underlying middleware and considers structured information at different levels; Devign proposes a new convolutional layer for classification, which improves the classification results; FUNDED uses a layer-by-layer high-speed gating unit to control the propagation of noisy data. BGNN4VD [18] utilizes a bidirectional gated graph neural network for feature vector representation of compound graphs with reverse edges, enabling information to be aggregated in reverse. Reveal also uses GGNN to extract graph feature vectors, using representation learning to improve detection performance.

Attention-based graph neural network methods are also widely used in vulnerability detection. VulSPG [45] introduced a triple attention mechanism of node attention mechanism, sentence attention mechanism, and subgraph attention mechanism for slice attribute graph to improve the ability of node information aggregation. ACGVD [46] designed a node-level attention mechanism and a path-level attention mechanism for data control flow graphs to improve the classification performance. IvDetect [34] uses PDG to represent the code structure, then extracts sub-AST and nodes from PDG, and finally uses Graph Convolution Network with feature-attention for feature extraction and classification.

8 CONCLUSION

This paper systematically studies a vulnerability detection model based on Graph Attention Network and Metric Learning to discover real-world code vulnerabilities effectively. GSM includes three stages graph feature extraction, Sampling, and Metric Learning. It solves the following three problems: 1) Inability to extract syntactic and semantic information sufficiently and effectively, 2) Data unbalance, 3) Inability to distinguish features related to vulnerabilities. We verify the performance of the multi-head attention mechanism-based graph attention network, sampling method, and Metric Learning method on the static code vulnerability detection task. When contrasted to the state-of-the-art approach Reveal, GSM has higher performance and can more effectively find vulnerable functions in static code. In the future, we will combine multiple project datasets for Semi-Supervised Adversarial Transfer Learning for the problem that labels are not easy to label, and finally complete the task of vulnerability detection across projects (projects under the same or different programming languages).

ACKNOWLEDGMENTS

This work was funded by National Key R&D Program of China (No.2020YFB1708600) and Foundation of Guizhou Provincial Key Laboratory of Public Big Data (No.2017BDKFJJO15, No.2018BDKFJJO08, No.2018BDKFJJO20, No.2018BDKFJJO21).

REFERENCES

- [1] S. Wu, "Review and outlook of information security vulnerability analysis," *Chinese Journal of Tsinghua University (Science and Technology)*, vol. 49, no. s2, pp. 2065–2072, 2009.
- [2] H. Booth, D. Rike, G. A. Witte et al., "The national vulnerability database (nvd): Overview," 2013.

- [3] B. Chess and M. Gershefske, "Rough auditing tool for security. google code," 2013.
- [4] D. A. Ramos and D. Engler, "{Under-Constrained} symbolic execution: Correctness checking for real code," in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 49–64.
- [5] T. Xie, N. Tillmann, J. De Halleux, and W. Schulte, "Fitness-guided path exploration in dynamic symbolic execution," in *2009 IEEE/IFIP International Conference on Dependable Systems & Networks*. IEEE, 2009, pp. 359–368.
- [6] A. Humphries, K. Cating-Subramanian, and M. K. Reiter, "Tase: Reducing latency of symbolic execution with transactional memory," *arXiv preprint arXiv:1912.12363*, 2019.
- [7] M. Sutton, A. Greene, and P. Amini, *Fuzzing: brute force vulnerability discovery*. Pearson Education, 2007.
- [8] S. Gan, C. Zhang, X. Qin, X. Tu, K. Li, Z. Pei, and Z. Chen, "Collafl: Path sensitive fuzzing," in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 679–696.
- [9] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, "Automated vulnerability detection in source code using deep representation learning," in *2018 17th IEEE international conference on machine learning and applications (ICMLA)*. IEEE, 2018, pp. 757–762.
- [10] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "Vuldeepecker: A deep learning-based system for vulnerability detection," *arXiv preprint arXiv:1801.01681*, 2018.
- [11] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "Sysevr: A framework for using deep learning to detect software vulnerabilities," *IEEE Transactions on Dependable and Secure Computing*, 2021.
- [12] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, " μ vuldeepecker: A deep learning-based system for multiclass vulnerability detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2224–2236, 2019.
- [13] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Y. Zhang, Z. Chen, and D. Li, "Vuldeeloctor: A deep learning-based system for detecting and locating software vulnerabilities," *IEEE Transactions on Dependable and Secure Computing*, 2021.
- [14] G. Lin, J. Zhang, W. Luo, L. Pan, Y. Xiang, O. De Vel, and P. Montague, "Cross-project transfer representation learning for vulnerable function discovery," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 7, pp. 3289–3297, 2018.
- [15] G. Lin, J. Zhang, W. Luo, L. Pan, O. De Vel, P. Montague, and Y. Xiang, "Software vulnerability discovery via learning multi-domain knowledge bases," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2469–2485, 2019.
- [16] S. Liu, G. Lin, Q.-L. Han, S. Wen, J. Zhang, and Y. Xiang, "Deepbalance: Deep-learning and fuzzy oversampling for vulnerability detection," *IEEE Transactions on Fuzzy Systems*, vol. 28, no. 7, pp. 1329–1343, 2019.
- [17] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet," *IEEE Transactions on Software Engineering*, 2021.
- [18] S. Cao, X. Sun, L. Bo, Y. Wei, and B. Li, "Bgnn4vd: constructing bidirectional graph neural-network for vulnerability detection," *Information and Software Technology*, vol. 136, p. 106576, 2021.
- [19] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.
- [20] H. Wang, G. Ye, Z. Tang, S. H. Tan, S. Huang, D. Fang, Y. Feng, L. Bian, and Z. Wang, "Combining graph-based learning with automated data collection for code vulnerability detection," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1943–1958, 2020.
- [21] G. Ye, Z. Tang, H. Wang, D. Fang, J. Fang, S. Huang, and Z. Wang, "Deep program structure modeling through multi-relational graph-based learning," in *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques*, 2020, pp. 111–123.
- [22] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, "Gated graph sequence neural networks," *arXiv preprint arXiv:1511.05493*, 2015.
- [23] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, "code2vec: Learning distributed representations of code," *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, pp. 1–29, 2019.
- [24] N. D. Bui, Y. Yu, and L. Jiang, "Infercode: Self-supervised learning of code representations by predicting subtrees," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1186–1197.
- [25] Y. Zhuang, S. Suneja, V. Thost, G. Domeniconi, A. Morari, and J. Laredo, "Software vulnerability detection via deep learning over disaggregated code graph representation," *arXiv preprint arXiv:2109.03341*, 2021.
- [26] Z. Yu, R. Cao, Q. Tang, S. Nie, J. Huang, and S. Wu, "Order matters: semantic-aware neural networks for binary code similarity detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 01, 2020, pp. 1145–1152.
- [27] S. M. Ghaffarian and H. R. Shahriari, "Neural software vulnerability analysis using rich intermediate graph representations of programs," *Information Sciences*, vol. 553, pp. 189–207, 2021.
- [28] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu et al., "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.
- [29] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, pp. 590–604.
- [30] X. Duan, J. Wu, S. Ji, Z. Rui, T. Luo, M. Yang, and Y. Wu, "Vul-sniper: Focus your attention to shoot fine-grained vulnerabilities," in *IJCAI*, 2019, pp. 4665–4671.
- [31] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
- [32] R. Rabheru, H. Hanif, and S. Maffei, "A hybrid graph neural network approach for detecting php vulnerabilities," *arXiv preprint arXiv:2012.08835*, 2020.
- [33] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1532–1543.
- [34] Y. Li, S. Wang, and T. N. Nguyen, *Vulnerability Detection with Fine-Grained Interpretations*. New York, NY, USA: Association for Computing Machinery, 2021, p. 292–303.
- [35] G. Lin, W. Xiao, J. Zhang, and Y. Xiang, "Deep learning-based vulnerable function detection: A benchmark," in *International Conference on Information and Communications Security*. Springer, 2019, pp. 219–232.
- [36] Q. Le and T. Mikolov, "Distributed representations of sentences and documents," in *International conference on machine learning*. PMLR, 2014, pp. 1188–1196.
- [37] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "Deepwukong: Statically detecting software vulnerabilities using deep graph neural network," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 3, pp. 1–33, 2021.
- [38] C. Manning, P. Raghavan, and H. Schütze, "Introduction to information retrieval," *Natural Language Engineering*, vol. 16, no. 1, pp. 100–103, 2010.
- [39] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [40] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.
- [41] R. Hadsell, S. Chopra, and Y. LeCun, "Dimensionality reduction by learning an invariant mapping," in *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, vol. 2. IEEE, 2006, pp. 1735–1742.
- [42] W. Chen, X. Chen, J. Zhang, and K. Huang, "Beyond triplet loss: a deep quadruplet network for person re-identification," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 403–412.
- [43] C. Mao, Z. Zhong, J. Yang, C. Vondrick, and B. Ray, "Metric learning for adversarial robustness," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [44] S. Mitra, L. Avra, and E. J. McCluskey, "Scan synthesis for one-hot signals," in *Proceedings International Test Conference 1997*. IEEE, 1997, pp. 714–722.
- [45] W. Zheng, Y. Jiang, and X. Su, "Vulspg: Vulnerability detection based on slice property graph representation learning," *arXiv preprint arXiv:2109.02527*, 2021.
- [46] M. Li, C. Li, S. Li, Y. Wu, B. Zhang, and Y. Wen, "Acgvd: Vulnerability detection based on comprehensive graph via graph neural network with attention," in *International Conference on Information and Communications Security*. Springer, 2021, pp. 243–259.
- [47] W. An, L. Chen, J. Wang, G. Du, G. Shi, and D. Meng, "Avdhram: Automated vulnerability detection based on hierarchical represen-

- tation and attention mechanism," in *2020 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom)*. IEEE, 2020, pp. 337–344.
- [48] D. Hin, A. Kan, H. Chen, and M. A. Babar, "Linevd: Statement-level vulnerability detection using graph neural networks," *arXiv preprint arXiv:2203.05181*, 2022.
 - [49] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: synthetic minority over-sampling technique," *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.
 - [50] M. Zeng, B. Zou, F. Wei, X. Liu, and L. Wang, "Effective prediction of three common diseases by combining smote with tomesk links technique for imbalanced medical data," in *2016 IEEE International Conference of Online Analysis and Computing Science (ICOACS)*, 2016, pp. 225–228.
 - [51] M. A. H. Farquard and I. Bose, "Preprocessing unbalanced data using support vector machine," *Decision Support Systems*, vol. 53, no. 1, pp. 226–233, 2012.
 - [52] L. Van der Maaten and G. Hinton, "Visualizing data using t-sne." *Journal of machine learning research*, vol. 9, no. 11, 2008.
 - [53] V. Okun, A. Delaitre, P. E. Black *et al.*, "Report on the static analysis tool exposition (sate) iv," *NIST Special Publication*, vol. 500, p. 297, 2013.
 - [54] Y. Zheng, S. Pujar, B. Lewis, L. Buratti, E. Epstein, B. Yang, J. Laredo, A. Morari, and Z. Su, "D2a: A dataset built for ai-based vulnerability detection methods using differential analysis," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2021, pp. 111–120.
 - [55] J. Platt, "Sequential minimal optimization: A fast algorithm for training support vector machines," 1998.
 - [56] P. Wang, C. Shen, N. Barnes, and H. Zheng, "Fast and robust object detection using asymmetric totally corrective boosting," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 23, no. 1, pp. 33–46, 2011.
 - [57] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
 - [58] K. Xu, C. Li, Y. Tian, T. Sonobe, K.-i. Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *International Conference on Machine Learning*. PMLR, 2018, pp. 5453–5462.
 - [59] J. Zhang, X. Shi, J. Xie, H. Ma, I. King, and D.-Y. Yeung, "Gaan: Gated attention networks for learning on large and spatiotemporal graphs," *arXiv preprint arXiv:1803.07294*, 2018.
 - [60] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," *Advances in neural information processing systems*, vol. 27, 2014.
 - [61] E. Yahav, "From programs to interpretable deep models and back," in *International Conference on Computer Aided Verification*. Springer, 2018, pp. 27–37.
 - [62] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A novel neural source code representation based on abstract syntax tree," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 783–794.
 - [63] T. Ben-Nun, A. S. Jakobovits, and T. Hoefler, "Neural code comprehension: A learnable representation of code semantics," *Advances in Neural Information Processing Systems*, vol. 31, pp. 3588–3600, 2018.